

HPC Configuration 2021

Rohit Goswami · 2021-05-09 · workflow · projects · hpc

A post on configuring a user account for heavy interactive HPC usage with `Spack`, `Nix` and self-hosted applications.

Background

I have in the past written a few posts about configuring my HPC setup. Since the [Research and University Network of Iceland hf. \(RHnet\)](#) recently recieved funding for a new machine, called Elja [^fn:1], I decided to jot down my notes from my setup as part of the user interaction team. Without abusing the ability to bend the ear of the system administrators, I decided to document a user-facing workflow. All the commands here assume an interactive session (example in `slurm`):

```
# 26 should be the number of processors on a node
srun -n 26 --pty /bin/bash
```

Rust and Utilities

`rustup` is a fantastic way to get `cargo` up and running and immediately allows for the installation of many relevant command line utilities.

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source $HOME/.cargo/env
```

The ones I use mostly are enumerated in Table 1.

Table 1: Language bindings and tools

Utility	Rust Alternative	Rationale
<code>cat</code>	<code>bat</code>	File reader with syntax highlighting
<code>top</code>	<code>bottom</code>	Process management
<code>ls</code>	<code>exa</code>	Listing things with colors
<code>ps</code>	<code>procs</code>	Better looking process lists
<code>grep</code>	<code>ripgrep</code>	Search recursively
<code>find</code>	<code>fd-find</code>	Find things
<code>sed</code>	<code>sd</code>	Search and replace things
<code>time</code>	<code>hyperfine</code>	Better benchmarking with stats
<code>man</code>	<code>tealdeer</code>	A fast <code>tldr</code> page viewer
<code>fzf</code>	<code>skim</code>	A fuzy finder
<code>-</code>	<code>navi</code>	Personalized cheatsheets
<code>cd / z</code>	<code>zoxide</code>	Directory jumping aid
<code>gibo</code>	<code>bliss</code>	<code>gitignore</code> generation utility
<code>diff</code>	<code>delta</code>	Nicer diffs

Utility	Rust Alternative	Rationale
---------	------------------	-----------

<code>watch</code>	<code>watchexec</code>	Watch and execute
--------------------	------------------------	-------------------

<code>CTRL+R</code>	<code>mcfly</code>	Reverse search
---------------------	--------------------	----------------

These are trivial to obtain:

```
cargo install bottom ripgrep hyperfine skim mcfly
# The following need some system libraries (OpenSSL) and shall be dealt with later, or with micromamba
# bat exa fd-find tealdeer starship sdprocs zoxide bliss navi watchexec
```

Not immediately obvious, but [easy to update](#) as well.

```
cargo install cargo-update
cargo install-update --all
```

MicroMamba

We will set up a `micromamba` installation to get `git` and other base utilities.

```
mkdir -p ~/.local/bin
export PATH=$HOME/.local/bin:$PATH
wget -qO- https://micromamba.snakepit.net/api/micromamba/linux-64/latest | tar -xvj bin/micromamba
mv bin/micromamba ~/.local/bin
rm -rf bin
micromamba shell init -s bash -p $HOME/.micromamba
. ~/.bashrc
```

Now we can use `mamba` to get a temporary interactive environment.

```
# Spack requirements
PKGS="git patch curl make tar gzip unzip bzip2 xz zstd subversion lsof m4 openssl pkg-config"
mkdir ~/.mamba_venvs
micromamba create -p ~/.mamba_venvs/intBase $PKGS
micromamba activate ~/.mamba_venvs/intBase
```

What we need `micromamba` for is really the `openssl` library.

Configuration

A simple `.condarc` will suffice for the above.

```
channels:
  - conda-forge
  - defaults
channel_priority: disabled
```

GitHub Client

We can grab the [GitHub client binary](#) for interacting with GitHub for now. Configuring and using this spares the necessity of explaining SSH authentication.

```
ghcliVer=1.13.1
DST=~/.local
wget https://github.com/cli/cli/releases/download/v${ghcliVer}/gh_${ghcliVer}_linux_amd64.tar.gz
tar xf "gh_${ghcliVer}_linux_amd64.tar.gz"
mkdir -p ~/.local/
cd "gh_${ghcliVer}_linux_amd64"
# Merges directories
# From: https://unix.stackexchange.com/a/603209/246530
find -type d -exec mkdir -vp "$DST"/{} \; -or -exec mv -nv {} "$DST"/{} \;
cd ../
rm -rf "gh_${ghcliVer}_linux_amd64" "gh_${ghcliVer}_linux_amd64.tar.gz"
```

Now we can set this up.

```
gh auth login # Follow the instructions
```

The rest of the post assumes `gh` is present.

Dotfiles

Some of this post is simplified by using [my Dotfiles](#) with `bombadil`:

```
cargo install toml-bombadil
mkdir -p $HOME/Git/Github
cd ~/Git/Github
gh repo clone HaoZeke/Dotfiles
cd Dotfiles
git checkout bombadil
bombadil install -c bombadil.toml
```

We will need to make some directories since `bombadil` does not currently handle this. Make changes to the variables as required and then link them.

```
mkdir -p ~/.config/nvim ~/.config/alacritty/ \
~/.spack/
bombadil link -p hpc elja # Combining profiles (variables)
```

Spack

For setting multi compiler workflows on the HPC, `spack` is a great tool. It is also well documented and easy to add to. Rather than setting up a hierarchical `Lmod` structure, we shall use a `spack environment`. Although we have already a `micromamba` environment, `spack` configurations are more suited to the HPC workflow; since they are more easily customized.

```
# Install
cd $HOME
gh repo clone spack/spack
```

Now we can start by getting a new compiler.

```
. ~/spack/share/spack/setup-env.sh
spack install gcc@9.3.0
spack compiler add $(spack location -i gcc@9.3.0)
spack install gcc@10.2.0 %gcc@9.3.0
spack compiler add $(spack location -i gcc@10.2.0)
spack activate interactive # After setting the environment
```

Environments

This step is not required if one uses my `dotfiles`.

```
# $SPACK_HOME//var/spack/environments/interactive/spack.yaml
spack:
  definitions:
    - compilers: ['gcc@10.2.0']
    - packages: [
      'zsh', 'tmux', 'git',
      'openssl', 'gmp', 'mpfr',
      'mpc', 'perl', 'boost', 'subversion',
      'zlib', 'cmake', 'bison', 'bdw-gc',
      'flex', 'pcre', 'jq', 'patch',
      'bzip2', 'sqlite', 'editline',
      'ncurses', 'texinfo', 'brotli'
    ]
  specs:
    - $compilers
    - 'subversion%gcc@9.3.0'
    - matrix:
      - [$packages]
      - [%compilers]
  view: true
```

Note that `subversion` depends on `serf` which currently does not build with `gcc@10.x.x`.

```
spackactivate interactive
spack install
```

Alternatively, one can just `spack load` packages of interest after `spack install`.

Cleaning Modules

This step is not required if one uses the `dotfiles`.

```

modules:
  enable::
    - lmod
  lmod:
    core_compilers:
      - 'gcc@10.2.0'
    hierarchy:
      - mpi
    hash_length: 0
    whitelist:
      - gcc
    blacklist:
      - '%gcc@10.2.0'
    all:
      filter:
        environment_blacklist:
          - "C_INCLUDE_PATH"
          - "CPLUS_INCLUDE_PATH"
          - "LIBRARY_PATH"
        environment:
          set:
            '{name}_ROOT': '{prefix}'
    openmpi:
      environment:
        set:
          SLURM_MPI_TYPE: pmi2
          OMPI_MCA_btl_openib_warn_default_gid_prefix: '0'
    projections:
      all:          '{name}/{version}'
      ^lapack:      '{name}/{version}-{^lapack.name}'

```

We can then rebuild the setup.

```
spack module lmod refresh --delete-tree -y
```

Nix

Portable

[nix-portable](#) is one of the best projects for working with local `nix` installations. Where it works, it works very simply:

```

wget https://github.com/DavHau/nix-portable/releases/download/v006/nix-portable
chmod +x nix-portable
mv nix-portable ~/.local/bin

```

Mamba

```

# Spack requirements
PKGS="git patch curl make tar gzip bzip2 xz zstd brotli openssl sqlite bdw-gc nlohmann_json editline libxml2
libxslt libseccomp bison flex lsof libarchive automake pkg-config"
micromamba create -p ~/.mamba_venvs/nixIns $PKGS
micromamba activate ~/.mamba_venvs/nixIns

```

It is best to have a clean, maximal `boost` .

```
git clone --recursive https://github.com/boostorg/boost.git
cd boost
git checkout tags/boost-1.73.0
./bootstrap.sh
./b2 headers
./b2
./b2 install --prefix=$HOME/.hpc/boost/boost-1.73.0
ml load boost/boost-1.73.0
```

Now we can try to compile a patched older `nix`.

```
export myprefix=$HOME/.hpc/nix/nix-boot
export nixdir=$HOME/.nix
./configure --enable-gc --prefix=$myprefix \
--with-store-dir=$nixdir/store --localstatedir=$nixdir/var \
\
--disable-seccomp-sandboxing --disable-doc-gen \
CPPFLAGS="-I$HOME/.mamba_venvs/nixIns/include" LDFLAGS="-L$HOME/.mamba_venvs/nixIns/lib -Wl,-R$HOME/.mamba_venvs/nixIns/lib"
make -j $(nproc)
make install
```

Spack

When the existing methods don't work, we can fall back to a more rustic method. We need some additional packages, which are not part of `spack` yet.

```
spack create https://github.com/anrieff/libcpuid/releases/download/v0.5.1/libcpuid-0.5.1.tar.gz
spack install libcpuid
```

```
mkdir -m 0755 ~/.nix
spack install nix storedir=$nixdir/store statedir=$nixdir/var doc=False sandboxing=False nfs=True
spack load nix
# Basic setup
mkdir -p ~/.nix/var/nix/profiles
nix-env --switch-profile .nix/var/nix/profiles/default
```

Now we can install this natively with `spack`.

```
mkdir -p $HOME/Git/Github
gh repo clone nixos/nixpkgs
cd nixpkgs
export NIX_PATH=$(pwd)
export nixdir=$HOME/.nix
export PATH=$(spack location -i nix)/bin/:$PATH:$nixdir/bin
export INCLUDE=$(spack location -i nix)/include/:$INCLUDE
export LD_LIBRARY_PATH=$(spack location -i nix)/lib:$LD_LIBRARY_PATH
export PKG_CONFIG_PATH=$(spack location -i nix)/lib/pkgconfig/:$PKG_CONFIG_PATH
nix-env -i nix -f $HOME/Git/Github/nixpkgs -j$(nproc) --keep-going --show-trace -v --cores $(nproc) 2>&1 | tee
nix-no-root.log
```

```

export BOOST_ROOT=$(spack location -i boost)
./configure --enable-gc \
  --prefix=$myprefix \
  --with-store-dir=$nixdir/store \
  --localstatedir=$nixdir/var \
  --with-boost=$BOOST_ROOT \
  --disable-seccomp-sandboxing \
  --disable-doc-gen \
  --includedir="$(spack location -i bdw-gc)" \
  --includedir="$(spack location -i bzip2)" \
  --includedir="$(spack location -i nlohmann-json)" \
  --libdir="$(spack location -i bzip2)" \
  --libdir="$(spack location -i bdw-gc)" \
  PKG_CONFIG=$(spack location -i pkg-config)/bin/pkg-config \
  OPENSSL_CFLAGS=-I$(spack location -i openssl)/include \
  OPENSSL_LIBS=-L$(spack location -i openssl)/lib \
  CPPFLAGS="-I$(spack location -i nlohmann-json)/include"
make -j $(nproc)
make install
ml load nix/user # Hooray!

```

Self hosted binaries

Code Server

The first of the selfhosted set is the [browser based IDE](#) inspired by Visual Studio Code.

```

export VERSION=3.12.0
mkdir -p ~/.local/lib ~/.local/bin
curl -fL "https://github.com/cdr/code-server/releases/download/v$VERSION/code-server-$VERSION-linux-
amd64.tar.gz" \
  | tar -C ~/.local/lib -xz
mv "$HOME/.local/lib/code-server-$VERSION-linux-amd64" "$HOME/.local/lib/code-server-$VERSION"
ln -s "$HOME/.local/lib/code-server-$VERSION/bin/code-server" "$HOME/.local/bin/code-server"
PATH="$HOME/.local/bin:$PATH"

```

This has the advantage of not needing any port forwarding (with `--link`):

```
code-server --link
```

Naturally this is often disabled by more security oriented SysAdmins. This can be circumvented with the same `ngrok` scheme as outlined earlier.

Conclusions

This is a work in progress, and would likely need to be updated ever so often, perhaps like a “State Of My Dots” or a yearly “HPC Configuration \$YEAR” or something like that.